

Store Frontend Architecture

Overview

The Yenkasa Store is not a React SPA. It is a multipage storefront composed of static HTML, CSS, and browser JavaScript, served by the main backend under `/store`. The store includes buyer flows, seller dashboards, admin dashboards, checkout, notifications, and account management.

Current implementation

- static assets live under `public/triciabales\_frontend`
- landing pages live under `public/triciabales\_frontend/landingFile`
- browser logic lives under `public/triciabales\_frontend/jsFiles`
- CSS lives under `public/triciabales\_frontend/cssFile`
- the backend maps clean `/store/...` URLs onto these static HTML files

```mermaid

flowchart TD

```
 Browser --> StorePages[Static HTML pages]
 StorePages --> StoreScripts[jsFiles/*.js]
 StoreScripts --> StoreProxy[/triciabales-api proxy routes]
 StoreProxy --> CoreBackend[Main Yenkasa API]
```

### ## Main storefront surfaces

- `index.html`: public store homepage
- buyer flows:
  - register
  - buyer login
  - cart
  - address
  - delivery
  - payment
  - thank-you
  - my orders
  - notifications
- seller flows:
  - seller login
  - seller dashboard
- admin flows:
  - admin
  - super admin

### ## Browser-side behavior

- heavy use of `localStorage` for current user, auth token, cart, checkout state, and seller session data
- homepage script handles:
  - category browsing
  - seller/product showcase
  - cart updates
  - support modal
  - navigation state
- seller dashboard script handles:
  - seller role checks
  - product upload/edit flows
  - order and payout sections
  - admin redirection rules

### ## Known issues

- the store uses many page-specific scripts rather than a shared application shell
- auth and role checks are repeated in browser JS
- state persistence relies heavily on local storage conventions

## ## Scaling concerns

- multipage vanilla JS becomes harder to evolve consistently as feature count rises
- shared behavior across buyer, seller, and admin pages is easy to duplicate
- browser-side token handling and page-level scripts increase maintenance overhead

## ## Future improvements

1. extract shared store utilities into a clearer client module layer
2. reduce duplicated auth/session checks across page scripts
3. evaluate whether the store should remain multipage or move toward a more unified frontend architecture